

Interface

1. Interface is a keyword which is used to define User Defined Data.
2. Interface is a special class which is fully abstracted.
3. Interface can be used to achieve the multiple Inheritances in java.
4. One Interface can extend one or more interfaces.
5. One class can implement one or more interfaces.

Syntax to declare interface

```
< Modifiers >interface< Interface Name >
{
    //Members
}
```

6. Interface can contain following members:-
 - ⇒ Public final static variables.
 - ⇒ Public abstract methods
 - ⇒ Public static inner classes
7. Variables declared in the interface are by default public, final and static. So it is unnecessary to use these modifiers for variables.
8. Methods declared in the interface are by default public and abstract. So it is Unnecessary to use these modifiers for methods.
9. You can use **implements** keyword to inherit the functionality of an interface.
10. When you write sub class of an interface then:-
 - ⇒ Subclass can use all final static variables.
 - ⇒ Subclass has to implement all the methods of implemented interfaces.
 - ⇒ When subclass is not implementing one or more methods then subclass must be declared as abstract.
11. You cannot instantiate interface i.e. object of interface cannot be created.
12. You can declare the reference variable of interface.
13. You cannot use following modifiers for interface variables.

Private	protected	volatile	transient
---------	-----------	----------	-----------
14. You cannot use following modifiers for interface methods.

Private	protected	static	final
Strictfp	synchronized	native	
15. You can not define following members in interface

Instance variable	static Methods	constructors	Initialization Blocks
-------------------	----------------	--------------	-----------------------
16. When you define an interface without any member than it is called as **MARKER INTERFACE** or **EMPTY INTERFACE** or **TAGGED INTERFACE**
17. Interface methods are by default public i.e. while implementing these methods remembers to use public modifier in subclasses.
18. When you are accessing interface member by interface reference variable which contains subclass object then that members should be available in interface.
19. Interface variable can be accessed with interface name also because of static.

20. When you are implementing two or more interfaces which contain common methods then in subclass you have to provide the implementation only once and single implementation will be valid for all the interfaces.
21. When you are implementing two or more interfaces which contain common variables then reference to that variable in subclass will be ambiguous. so to resolve ambiguity you have to refer that common variable with interfaces name.

Check following combination with program

Ex1 abstract interface Inter{ void m1(); } // valid	Ex2 interface Inter{ private void m1(); } // error:modifier private not allowed here
Ex2 interface Inter{ protected void m1(); } // error: modifier protected not allowed here	Ex4 interface Inter{ native void m1(); } // error: modifier native not allowed here
Ex5 interface Inter{ strictfp void m1(); } // error: modifier strictfp not allowed here	Ex6 interface Inter{ synchronized void m1(); } // error: modifier synchronized not allowed
Ex7 interface Inter{ final void m1(); } // error: modifier final not allowed here	Ex8 interface Inter{ static abstract void m1(); } // error: modifier static not allowed here
Ex9 interface Inter{ private int A=10; } // error: modifier private not allowed here	Ex10 interface Inter{ protected int A=10; } // error: modifier protected not allowed
Ex11 interface Inter{ transient int A=10; } // error: modifier transient not allowed here	Ex12 interface Inter{ volatile int A=10; } // error: modifier volatile not allowed
Ex13 interface Inter{ int A; } // error: = expected	Ex14 interface Inter <u>extends</u> Object{ void m1(); } // error: '{' expected

Ex15

```
interface Inter1{}
interface Inter2{}
interface Inter3 implements Inter1,Inter2{}
//error: '{' expected
```

Ex16

```
interface Inter1{}
interface Inter2{}
interface Inter3 extends Inter1,Inter2{}
//valid
```

Ex17

```
interface Inter1{}
interface Inter2{}
class Hai{}
class Hello implements Inter1,Inter2 extends hai{} // error: ':' expected
```

Ex18

```
interface Inter1{}
interface Inter2{}
class Hai{}
class Hello extends Inter1,Inter2 implements hai{} // valid
```

Ex19

```
interface Inter1{}
Inter1(){
{
System.out.println("Inter1 I.B");
}
Static {
System.out.println("Inter1 S.B");
}
//Invalid
```

Programs:-**OOPS402.java**

```
interface inter1{
void m1();
public abstract void m2();
int A=10;
public final int B=20;
}

class Hello extends inter1{
class oops932 {
public static void main(String[] args) {
System.out.println("Hello guys");
}
}
}
```

OOPS403.java

```
interface Inter1{
void m1();
public abstract void m2();
int A=10;
public final int B=20;
}

class Hello implements Inter1{
class oops933{
public static void main(String arg[]){
System.out.println("Hello guys");
}
}
}
```

OOPS404.java

```

interface Inter1{
    void m1();
    public abstract void m2();
    int A=10;
    public final int B=20;
}

abstract class Hello implements Inter1{
class OOPS404{
    public static void main(String arg[]){
        System.out.println ("Hello guys!");
    }
}

```

OOPS405.java

```

interface Inter1{
    void m1();
    public abstract void m2();
    int A=10;
    public final int B=20;
}

class Hello implements Inter1{
    void m1(){
        System.out.println("Hello->m1()");
    }
    public void m2(){
        System.out.println("Hello->m2()");
    }
}

class OOPS405{
    public static void main(String arg[]){
        System.out.println("Hello guys");
    }
}

```

OOPS406.java

```

interface Inter1{
    void m1();
    public abstract void m2();
    int A=10;
    public final int B=20;
}

class Hello implements Inter1{
    public void m1(){
        System.out.println("Hello->m1()" +A);
    }
    public void m2(){
        System.out.println("Hello->m2()" +B);
    }
}

class OOPS406{
    public static void main(String arg[]){
        // Inter1 ref= new Hello();
        Inter1 ref=null;
        ref =new Hello();
        ref.m1();
        ref.m2();
    }
}

```

OOPS407.java

```

interface Inter1{
    void m1();
    public abstract void m2();
    int A=10;
    public final int B=20;
}

abstract class Hello implements Inter1{
    public void m1(){
        System.out.println("Hello->m1() " +A);
    }
}

class Hai extends Hello{
    public void m2(){
        System.out.println("Hello->m2() " +B);
    }
    void m3(){
        System.out.println("Hello->m3()");
    }
}

class OOPS407{
    public static void main(String arg[]){
        Inter1 ref=new Hai();
        ref.m1();
        ref.m2();
        //ref.m3();
    }
}

```

OOPS408.java

```

interface Inter1{
void m1();
int A=10;
}
interface Inter2{
void m2();
int B=11;
}
class Hello implements Inter1,Inter2{
public void m1(){
System.out.println("Hello->m1()");
}
}

```

```

public void m2(){
System.out.println("Hello->m2()");
}
}
class OOPS408{
public static void main(String arg[]){
Hello h=new Hello();
System.out.println(h. A);
System.out.println(h. B);
h.m1();
h.m2();
}
}

```

Inter1.java

```

interface Inter1{
void m1();
void m2();
int A=10;
int B=20;
}

```

Inter2.java

```

interface Inter2{
void m2();
void m3();
int A=11;
int C=30;
}

```

OOPS409.java

```

class Hello implements Inter1,Inter2{
public void m1(){
System.out.println("Hello->m1()");
}
public void m2(){
System.out.println("Hello->m2()");
}
public void m3(){
System.out.println("Hello->m3()");
System.out.println(B);
System.out.println(C);
}
}
class OOPS409{
public static void main(String arg[]){
Hello h=new Hello();
h.m1();
h.m2();
h.m3();
}
}

```

OOPS410.java

```

class Hello implements Inter1,Inter2{
public void m1(){
System.out.println("Hello->m1()");
}
public void m2(){
System.out.println("Hello->m2()");
}
public void m3(){
System.out.println("Hello->m3()");
System.out.println(A);
System.out.println(B);
System.out.println(C);
}
}
class OOPS410{
public static void main(String arg[]){
Hello h=new Hello();
h.m1();
h.m2();
h.m3();
}
}

```

OOPS411.java

```

class Hello implements Inter1,Inter2{
    public void m1(){
        System.out.println("Hello->m1()");
    }
    public void m2(){
        System.out.println("Hello->m2()");
    }
    public void m3(){
        System.out.println("Hello->m3()");
        System.out.println(Inter1.A);
        System.out.println(Inter2.A);
        System.out.println(B);
        System.out.println(C);
    }
}

class OOPS411{
    public static void main(String arg[]){
        Hello h=new Hello();
        h.m1();
        h.m2();
        h.m3();
    }
}

```

OOPS412.java

```

class Hello implements Inter1,Inter2{
    public void m1(){
        System.out.println("Hello->m1()");
    }
    public void m2(){
        System.out.println("Hello->m2()");
    }
    public void m3(){
        System.out.println("Hello->m3()");
        System.out.println(Inter1.A);
        System.out.println(Inter2.A);
        System.out.println(B);
        System.out.println(C);
    }
}

class OOPS412{
    public static void main(String arg[]){
        Hello h=new Hello();
        Inter1 ref1=h;
        ref1.m1();
        ref1.m2();
        Inter2 ref2=h;
        ref2.m2();
        ref2.m3();
    }
}

```

OOPS413.java

```

class A{
    static int X=90;
}

class B extends A{
    void show(){
        System.out.println("B->show()");
        System.out.println(X);
        X=99;
    }
}

```

```

class C extends A{
    void display(){
        System.out.println("C->display()");
        System.out.println(X);
    }
}

class OOPS413{
    public static void main(String arg[]){
        B bobj=new B();
        bobj.show();
        C cobj=new C();
        cobj.display();
    }
}

```

Limitation of Multiple Inheritance : -**OOPS414.java**

```

interface Inter1{
    void show();
}

interface Inter2{
    int show();
}

class A implements Inter1{
    public void show(){ }
}

```

```

class B implements Inter2{
    public int show(){
        return 0;
    }
}

class C implements Inter1,Inter2{
    public void show(){ }
    public int show(){ }
}

```

```

OOPS415.java
interface Inter1{
void show();
}
class A {
public static void show(){
System.out.println("show method");
}
}
class B extends A implements Inter1{
public void show(){
}
}

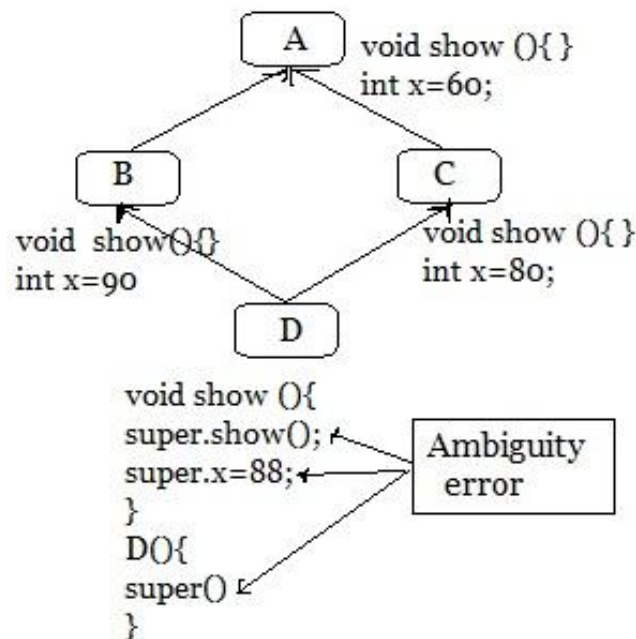
```

Program output /error and details:-

Prog No.	output	error and details
OOPS402	Compilation error	[error: no interface expected here] Refer page 145 point 9
OOPS403	Compilation error	[Error: Hello is not abstract and does not override abstract method m2() in Inter1] Refer page 145 point 10
OOPS404	Hello Guys!	Refer page 145 point 9 and 10.
OOPS405	Compilation error	[error: attempting to assign weaker access privileges; was public]
OOPS406	Hello->m1() 10 Hello->m2() 20	Refer page 145 point 11 and 12.
OOPS407	Hello->m1() 10 Hello->m2() 20	Refer page 145 point 11 and 12.
OOPS408	10 11 Hello->m1() Hello->m2()	Here, Interface is use to achieve the multiple Inheritances in java.
OOPS409	Hello->m1() Hello->m2() Hello->m3() 20 30	same""
OOPS410	Compilation error	[error: reference to A is ambiguous, both variable A in Inter1 and variable A in Inter2 match] In both class file variable A is declared.
OOPS411	Hello->m1() Hello->m2() Hello->m3() 10 11 20 30	For call to variable A from both file you can use class name as a reference with variable name. Refer page 148 and point 21.

Prog No.	output	error and details
OOPS412	Hello->m1() Hello->m2() Hello->m2() Hello->m3() 10 11 20 30	Refer page 147 point 17 and 18.
OOPS413	B->show() 90 C->display() 99	This is not interface.
OOPS414	Compilation error	Method return type is not different. It is limitation of Interface.
OOPS415	Compilation error	same""

Problem with multiple inheritance using classes:



When a class extends multiple classes then some members may common in super classes. When you refer those common member from the sub class then it result in ambiguity problem. When you create subclass object then first super class constructor should be invoked. When multiple super classes are there then which super class constructor will be invoked first.

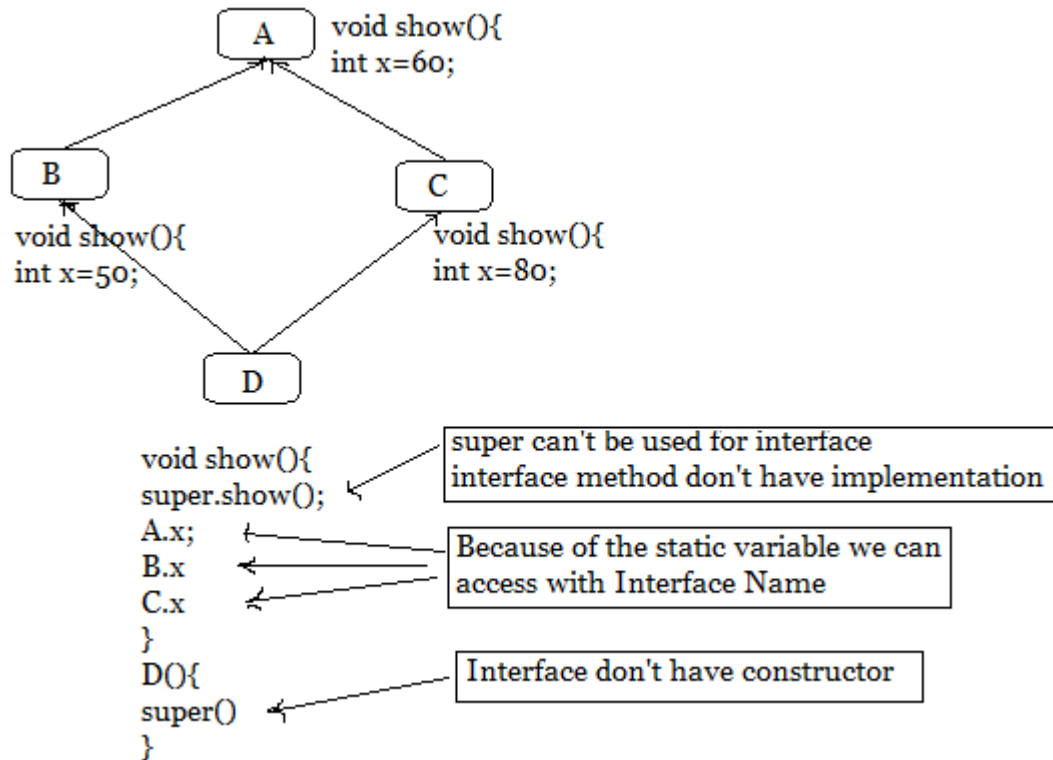
```

D ref=new D ();
ref. show();
System.out.println (ref. x);

```

Here, if A class has some instance variable than those will be inherited in B and C class. If D class is extending both B and C than 2 copies will be available for the same variable if you create object of D class than memory will be wastage.

Multiple Inheritance using Interface:



Example: -

OOPS416.java

```

abstract class OOPS416{
    public static void main(String arg[])
    {
        System.out.println("Hello guys");
    }
}
  
```

//Output : Hello guys

OOPS417.java

```

abstract class B{
    abstract void m1();
}
class A extends B{
    void m1(){
        System.out.println("Hello guys");
        super.m1();
    }
}
//Error: Cannot directly invoke the
abstract method m1() for the type B
  
```

OOPS418.java

```

class OOPS418{
    public static void main(String arg[]){
        A ref=new A();
        ref.show();
    }
}
interface Inter1{
    int xy=90;
}
  
```

```

}
class A implements Inter1{
    String xy="IND";
    void show(){
        System.out.println(xy);
        //System.out.println(super.xy);
        System.out.println(Inter1.xy);
    }
}
//Output: IND
9
  
```

OOPS419.java

```

class OOPS419{
public static void main(String arg[]){
A ref=new A();
ref.show();
}
interface Inter1{
int xy=90;
}
class B{
Boolean xy=true;
}

```

```

class A extends B implements Inter1{
String xy="IND";
void show(){
System.out.println(xy);
System.out.println(super.xy);
System.out.println(Inter1.xy);
}
}

```

```

//Output: IND
         true
         90

```

Different between Abstract and Interface:

Abstract	Interface
- Defining a class with abstract modifier. - Can define non –final/final variable. - Can define non-static /static variable. - Use any access modifier with members. - Define abstract /non-abstract method. - Can define non-static /static method. - Can define final/non-final method. - Can define static/non-static inner class. - Can define constructor/initialization block. - Cannot be used for multiple Inheritance. - You need to use extend keyword for defining subclass. - Can use executed if main method is available. - Can extend java.lang.object.	- Defining a class with interface keyword - Variable are by default final. - Variable are by default static. - By default member will be public, other access modifiers can't be used. - Method will be by default abstract. - Cannot define static method (because of abstract method). - Cannot define final method (because of abstract method). - Inner class will be default static. - Cannot define constructor/initialization block - Can be used for multiple Inheritance - You need to use implements keyword for defining subclass. - Cannot define main method. - Cannot extend java.lang.object

Write your note here.....

Compilation step to compile source file and generate the class file

- ❖ Compiler will parse the source file
 - ⇒ Read the source file
 - ⇒ Verify the syntax error.
- ❖ If any parsing error found then non of the .class file will be generated
 - ; is not used to end the statement.
 - } is not used to end of block
- ❖ If no parsing error found then compiler will verify the grammatical error for each class in the direction from top to bottom.
- ❖ If the class don't have grammatical error then class file will be generated.
- ❖ If any classes have grammatical error then classes available before the current class will be converted into class file but from the current class onward other class won't be converted into .class file.

Grammatical error:

- ❖ Variable you are using is not initialized.
- ❖ Non Static variable you are using from Static context.
- ❖ Re initialization the final variable **etc.**

You can verify Compiler process using -verbose

javac - verbose <Source file Name>.java

Miscellaneous

1. You cannot assign one variable to another reference without having any inheritance relationship between corresponding classes.

Example:

```
class Hello{ }
class Hai{ }
Hello hello=new Hello();
Hai hai=new Hai();
hello=hai;           //Invalid
hai=hello;           //Invalid
```

2. You can assign sub class object to super class type reference variable directly. The process is called as UPCASTING.

Example:

```
class Hello{ }
class Hai extends Hello{ }
Hello hello=new Hello();
Hai hai=new Hai();
hello=hai;           //Valid (Upcasting)
```

3. You cannot assign super class object to sub type reference variable directly. You can do it by using explicit type casting. This process is called as DOWNCASTING.

Example:

```
class Hello{ }
class Hai extends Hello{ }
Hello hello=new Hai();
Hai hai=hello;           //Invalid
Hai hai=(Hai)hello;      //Valid (down casting)
hai=hello;               //Invalid
```

4. When you perform down casting it may result in ClassCastException.

Example:

```
class A{ }
class B extends A{ }
class C extends B { }

A aobj=new A();
B bobj=new B();      //ClassCastException
C cobj=new C();      //ClassCastException

A aobj=new B();
B bobj=(B);          //Valid
C cobj=(C);          //ClassCastException

A aobj=new C();
B bobj=(B)aobj;      //ClassCastException
C cobj=(C)aobj;      //Valid
```

5. You can use instanceof operator before down casting to check whether required object is available in reference variable or not.
6. When you create the array of a class then that array can hold either same class object or sub class objects but not super class object.
7. When you create the array of an abstract class then that array can hold object of subclass of that abstract class.
8. When you create the array of an interface then that array can hold objects of subclass of the interface.
9. When you store object in array it may give ArrayStoreException.

Example:

```
class A{ }
class B extends A{ }
class C extends B { }
class D extends A{ }
A arr[]=new B[2];
arr[0]=new B();      //Valid
arr[0]=new C();      //Valid
arr[0]=new D();      //ArrayStoreException
```

Programs:-

A.java

```
class A{}
class B extends A{}
class C extends A{}
class D extends B{}
class E{}

```

OOPS420.java

```
class OOPS420{
    public static void main (String arg[]){
        D dobj=new D();
        A aobj=dobj;
        B bobj=aobj;
        D dobj=aobj;
    }
}

```

OOPS421.java

```
class OOPS421{
public static void main(String arg[]){
D dobj=new D();
A aobj=dobj;
B bobj=(B)aobj;
D dobj=(D)aobj;
System.out.println("Hello");
}
}
```

OOPS422.java

```
class OOPS422{
public static void main(String arg[]){
A aobj=new D();
C cobj=(C)aobj;
System.out.println("Hello");
}
}
```

OOPS423.java

```
class OOPS423{
public static void main(String arg[]){
A aobj=new D();
if(aobj instanceof C){
C cobj=(C)aobj;
}
System.out.println("Hello");
}
}
```

OOPS424.java

```
class OOPS424{
public static void main(String arg[]){
A aobj=new D();
E eobj=(E)aobj;
}
System.out.println("Hello");
}
}
```

OOPS425.java

```
class OOPS425{
public static void main(String arg[]){
A aobj=new D();
System.out.println(aobj instanceof A);
System.out.println(aobj instanceof B);
System.out.println(aobj instanceof C);
System.out.println(aobj instanceof D);
}
}
```

OOPS426.java

```
class OOPS426{
public static void main(String arg[]){
A aobj=new D();
System.out.println(aobj instanceof E);
}
}
```

OOPS427.java

```
class OOPS427{
public static void main(String arg[]){
A aobj=new D();
D dobj=new D();
B bobj=new B();
C cobj=new C();
System.out.println(aobj==dobj);
System.out.println(aobj==bobj);
System.out.println(aobj==cobj);
}
}
```

OOPS428.java

```
class OOPS428{
public static void main(String arg[]){
A aobj=new D();
E eobj=new E();
System.out.println(aobj==eobj);
B bobj=new B();
C cobj=new C();
System.out.println(bobj==cobj);
}
}
```

Inter1.java

```
interface Inter1 extends Object{ }
```

OOPS429.java

```
interface Inter1{
void m1();
}
class Hello implements Inter1{
public void m1(){
System.out.println("Hello->m1()");
}
public void m2(){
System.out.println("Hello->m2()");
}}
class OOPS429{
public static void main (String arg[]){
Inter1 in1=new Hello();
in1.m1();
}}
```

OOPS430.java

```
interface Inter1{
void m1();
}
class Hello implements Inter1{
public void m1(){
System.out.println("Hello->m1()");
}
public void m2(){
System.out.println("Hello->m2()");
}}
class OOPS430{
public static void main (String arg[]){
Inter1 in1=new Hello();
in1.m2();
}}
```

OOPS431.java

```
interface Inter1{
void m1();
}
class Hello implements Inter1{
public void m1(){
System.out.println("Hello->m1()");
}
public void m2(){
System.out.println("Hello->m2()");
}}
class OOPS431{
public static void main(String arg[]){
Inter1 in1=new Hello();
Object obj=in1;
System.out.println(in1.toString());
}}
```

OOPS432.java

```
interface Inter1{
class A implements Inter1{ }
class B extends A{}
class C extends A{}

class OOPS432{
public static void main(String arg[]){
Inter1 arr[]=null;
arr=new A[3];
arr[0]=new A();
arr[1]=new B();
arr[2]=new C();
for(int i=0;i<arr.length;i++)
System.out.println(arr[i]);
}
}
```

OOPS433.java

```
interface Inter1{
class A implements Inter1{ }
class B extends A{}
class C extends A{}
class OOPS433{
public static void main(String arg[]){
Inter1 arr[]=null;
arr=new B[3];
arr[0]=new A();
for(int i=0;i<arr.length;i++)
System.out.println(arr[i]);
}
}
```

OOPS434.java

```
interface Inter1{
class A implements Inter1{ }
class B extends A{}
class C extends A{}
class OOPS434{
public static void main(String arg[]){
Inter1 arr[]=new B[3];
arr[0]=new B();
arr[1]=new B();
arr[2]=new C();
for(int i=0;i<arr.length;i++)
System.out.println(arr[i]);
}
}
```

Program output /error and details:-

When you compile the A.java file then A, B, C, D and E class file will be created.

Prog No.	Output	Error and Details
OOPS420	Compilation Error	[Error: incompatible types B bobj= a obj;] We are not creating object for B and storing aobject in B
OOPS421	Compilation Error	[Error: variable dobj is already defined in method main(String[]) D dobj=(D)aobj;]
OOPS422	Runtime Error	[Error: ClassCastException: D cannot be cast to C]
OOPS423	Hello	1 st checking C is member of class A or not if yes then type caste otherwise not type caste.
OOPS424	Compilation Error	[Error: illegal start of type] we cannot type caste like that.
OOPS425	true true false true	Checking all the member is belong to that class or not. If yes then it return true otherwise false.
OOPS426	Compilation Error	[Error: inconvertible types (aobj instanceof E); required: E found: A] first create object for E.
OOPS427	false false false	[Error: incomparable types: A and E/B and C] Here, We are verifying that after creating the object that object can refer the same address or different.
OOPS428	Compilation Error	[Error: incomparable types: A and E/ B and C] Here, you are not able create subclass for E.
OOPS429	Hello->m1()	you are accessing only m1 () method.
OOPS430	Compilation Error	[Error: cannot find symbol in1.m2();]you can't access m2 directly.
OOPS431	Hello@19189e1	to String method is return address of the object with class name.
OOPS432	A@e53108 B@f62373 C@19189e1	Refer page159 point6, 7, 8 and 9
OOPS433	Runtime error	[ArrayStoreException] Refer page159 point6, 7, 8 and 9
OOPS434	Runtime error	[ArrayStoreException] Refer page159 point6, 7, 8 and 9

Write your note here.....